

## OSSP Skill Week 4

### Task 1: Tokenizing Shell Input.

#### Aim

To split a shell command into tokens and identify the individual arguments. Program

#### Program:

```
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 20

int main(void)
{
    char input[200];
    char *tokens[MAX_TOKENS];
    char *token;
    int count = 0;

    printf("myShell> ");
    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Empty command.\n");
        return 0;
    }

    /* Split on spaces, tabs and newlines */
    token = strtok(input, " \t\n");
    while (token != NULL && count < MAX_TOKENS) {
        tokens[count++] = token;
        token = strtok(NULL, " \t\n");
    }

    if (count == 0) {
        printf("Empty command.\n");
        return 0;
    }

    printf("\n--- Parsed Arguments ---\n");
    for (int i = 0; i < count; i++)
        printf("Argument %d: %s\n", i + 1, tokens[i]);

    printf("Tokenization completed successfully.\n");
    return 0;
}
```

#### Output

```
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc parser.c -o parser
● hrushika@Hrushikas-MacBook-Air pjsdj % ./parser
myShell> hello

--- Parsed Arguments ---
Argument 1: hello
Tokenization completed successfully.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

#### Observation

The command was tokenized successfully and the individual arguments were displayed.

## Task 2: Single Quote Handling

### Aim

To handle single quotes in shell commands and detect unmatched quotes.

### Program

```
single_quote.c:7:1: warning: implicit declaration of function 'main' is invalid in C99 [-Wimplicit-function-declaration]
#include <stdio.h>
#include <string.h>

int main(void)
{
    char input[200];
    int quotes = 0;

    printf("myShell> ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    /* Count single-quote characters */
    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\'')
            quotes++;
    }

    if (quotes % 2 != 0) {
        printf("Syntax Error: Unmatched single quote.\n");
        return 1;
    }

    printf("Parsing successful.\n");
    printf("Literal content preserved.\n");
    return 0;
}
```

### Output

```
hrushika@hrushikas-MacBook-Air pjsdj % gcc single_quote.c -o single_quote
hrushika@Hrushikas-MacBook-Air pjsdj % ./single_quote
myShell> hello
Parsing successful.
Literal content preserved.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

### Observation

Single-quoted strings were handled successfully and unmatched quotes were detected

## Task 3: Double Quote Handling

### Aim

To handle double quotes in shell commands, preserve spaces inside them and detect unmatched double quotes.

### Program

```
double_quote.c > main(void)
int arg = 1;

while (*start != '\0' && *start != '\n') {
    while (*start == ' ')
        start++;

    if (*start == '\0' || *start == '\n')
        break;

    if (*start == '"') {
        /* Quoted argument: keep everything up to the closing quote */
        start++;
        char *end = strchr(start, '"');
        if (end == NULL)
            break;
        *end = '\0';
        printf("Argument %d: %s\n", arg++, start);
        start = end + 1;
    } else {
        /* Plain argument: read up to the next space */
        char *end = start;
        while (*end != ' ' && *end != '\n' && *end != '\0')
            end++;

        char temp = *end;
        *end = '\0';
        printf("Argument %d: %s\n", arg++, start);

        if (temp == '\0' || temp == '\n')
            break;
        start = end + 1;
    }
}

return 0;
}
```

### Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % gcc double_quote.c -o double_quote
hrushika@Hrushikas-MacBook-Air pjsdj % ./double_quote
myShell> pwd
--- Parsed Arguments ---
Argument 1: pwd
hrushika@Hrushikas-MacBook-Air pjsdj %
```

### Observation

The text inside double quotes was treated as a single argument with its inner space preserved, and a missing closing quote was detected as an error.

### Result

Double-quoted strings were parsed successfully and syntax errors were detected.

## Task 4: Escape Sequence Processing

### Aim

To process backslash escapes in shell input so that escaped spaces and special symbols are kept as literal characters.

### Program

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char input[200];

    printf("myShell> ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    printf("--- Parsed Input ---\n");

    for (int i = 0; input[i] != '\0' && input[i] != '\n'; i++) {
        /* A backslash makes the next character literal */
        if (input[i] == '\\' && input[i + 1] != '\0' && input[i + 1] != '\n')
            i++;
        putchar(input[i]);
    }

    printf("\nParsing completed successfully.\n");
    return 0;
}
```

### Output

```
Argument 1: pwd
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc escape1.c -o escape1
● hrushika@Hrushikas-MacBook-Air pjsdj % ./escape1
myShell> pwd
--- Parsed Input ---
pwd
Parsing completed successfully.
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

### Observation

The backslash was recognized as an escape character, and the escaped space and the @ symbol were kept as part of the input.

### Result

Escape sequences were handled successfully and the original characters were preserved.

## Task 5: Creating Child Processes and Executing Programs

### Aim

To create a child process and execute a command with its arguments using `fork()` and `exec()`.

### Program

```
execute.c:1 main(int, char **)
~/Downloads/pjsdj/execute.c
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("Usage: %s <command> [arguments...]\n", argv[0]);
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        /* Child: run the command with its arguments */
        printf("Child Process PID: %d\n", getpid());
        execvp(argv[1], &argv[1]);
        perror("exec failed");
        exit(1);
    } else {
        int status;

        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        if (waitpid(pid, &status, 0) < 0) {
            perror("waitpid");
            return 1;
        }

        if (WIFEXITED(status))
            printf("Child exited with status: %d\n", WEXITSTATUS(status));
    }

    return 0;
}
```

### Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % ./execute
Usage: ./execute <command> [arguments...]
hrushika@Hrushikas-MacBook-Air pjsdj % ./execute ls -l
Parent Process PID: 58000
Created Child PID: 58001
Child Process PID: 58001
total 1608
-rw-r--r--@ 1 hrushika  staff   76 Oct  5 10:08 File1.txt
-rw-r--r--@ 1 hrushika  staff   76 Oct  5 10:09 File2.txt
-rwxr-xr-x@ 1 hrushika  staff 33928 Oct  6 16:10 a.out
-rwxr-xr-x@ 1 hrushika  staff 33696 Oct  6 16:35 double_quote
-rw-r--r--@ 1 hrushika  staff 1481 Oct  6 16:35 double_quote.c
-rwxr-xr-x@ 1 hrushika  staff 33432 Oct  6 16:13 escape
-rw-r--r--@ 1 hrushika  staff   150 Oct  6 16:12 escape.c
-rwxr-xr-x@ 1 hrushika  staff 33688 Oct  6 16:42 escape1
-rw-r--r--@ 1 hrushika  staff   533 Oct  6 16:42 escape1.c
-rwxr-xr-x@ 1 hrushika  staff 33688 Oct  6 16:46 execute
-rw-r--r--@ 1 hrushika  staff   912 Oct  6 16:46 execute.c
-rwxr-xr-x@ 1 hrushika  staff 33944 Oct  5 09:39 fork.exec
-rw-r--r--@ 1 hrushika  staff   935 Oct  5 09:39 fork.exec.c
-rwxr-xr-x@ 1 hrushika  staff 33864 Oct  5 09:46 main_loop
-rw-r--r--@ 1 hrushika  staff   618 Oct  5 09:46 main_loop.c
-rwxr-xr-x@ 1 hrushika  staff 33688 Oct  6 16:28 parser
-rw-r--r--@ 1 hrushika  staff   806 Oct  6 16:28 parser.c
-rwxr-xr-x@ 1 hrushika  staff 33648 Oct  6 16:33 single_quote
-rw-r--r--@ 1 hrushika  staff   546 Oct  6 16:32 single_quote.c
-rwxr-xr-x@ 1 hrushika  staff 33928 Oct  6 16:10 sk3t3
-rw-r--r--@ 1 hrushika  staff 1637 Oct  6 16:10 sk3t3.c
-rwxr-xr-x@ 1 hrushika  staff 33768 Oct  5 10:06 task2
-rw-r--r--@ 1 hrushika  staff   875 Oct  5 10:06 task2.c
-rwxr-xr-x@ 1 hrushika  staff 33928 Oct  5 10:12 task3
-rw-r--r--@ 1 hrushika  staff 1638 Oct  5 10:11 task3.c
-rwxr-xr-x@ 1 hrushika  staff 33944 Oct  6 16:19 task5
-rw-r--r--@ 1 hrushika  staff 3852 Oct  6 16:19 task5.c
-rwxr-xr-x@ 1 hrushika  staff 53960 Oct  6 16:20 task5_asan
drwxr-xr-x@ 3 hrushika  staff   96 Oct  6 16:20 task5_asan.dSYM
```

**Observation**

The parent created a child process, and the child executed the given command. A valid command exited with status 0, while an invalid argument produced an error message and a non-zero exit status (2).